

Práctica 3. Preparación de Datos

Tratamiento de Datos, Grado en Ciencia de Datos - UV

Contents

Introducción	1
Librería <code>tidyr</code> : organización de datos en tablas	2
Librería <code>magrittr</code> : cecí n'est pas une pipe	4
Ejemplo de visualización	4
Las cabeceras de las columnas son variables en lugar de nombres de variables	5
Varias variables en una misma columna	6
Variables almacenadas tanto en filas como en columnas	7
Varias unidades de observación en una sola tabla	7
Una única unidad de observación almacenada en varias tablas	7
Otro conjunto messy...	8
Entrega de ejercicios	8

Introducción

El objetivo de esta práctica es preparar los datos de una manera estructurada. Para ello vamos a seguir las recomendaciones del artículo de Hadley Wickham, *Tidy Data*, donde nos muestra una forma de estructurar (u organizar) los datos, de manera que se faciliten posteriores cadenas de procesado (visualización, obtención de datos estadísticos, modelización, etc.).

Decimos que un conjunto de datos está estructurado u organizado (se usa el término *tidy* en inglés) cuando cumple estas tres condiciones:

1. Cada variable está en una columna.
2. Cada observación está en una fila.
3. Cada unidad de observación está en una tabla.

Por el contrario, si no se cumple cualquiera de estas tres condiciones decimos que los datos están desordenados (*messy*).

Parece fácil, pero el problema viene al definir exactamente qué es lo que entendemos por variables y observaciones.

- Una variable (columna) contiene los valores correspondientes a un atributo (ej. altura, tiempo, etc.).
- Una observación (fila) contiene los valores medidos sobre la misma unidad (una persona, una fecha, una muestra etc.) a lo largo de los atributos.

Vamos a ver un ejemplo con el dataset IRIS.

```
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

¿Es un conjunto *tidy*?

- ¿Cada variable está en una columna? Depende de lo que definamos como variable. Según hemos visto, una variable debería contener todos los valores que miden un atributo. En este caso el atributo **Width** aparece en dos columnas, **Sepal.Width** y **Petal.Width**. Lo mismo ocurre con **Length**. También las partes de la flor: **Sepal** y **Petal** aparecen en varias columnas.
- ¿Cada fila es una única observación?

Veamos que aspecto tiene el conjunto IRIS una vez estructurado:

```
load('data/iris_tidy.Rdata')
head(iris.tidy)
```

	Species	Part	Measure	Value
1	setosa	Sepal	Length	5.1
2	setosa	Sepal	Length	4.9
3	setosa	Sepal	Length	4.7
4	setosa	Sepal	Length	4.6
5	setosa	Sepal	Length	5.0
6	setosa	Sepal	Length	5.4

Como se puede observar, en la versión *tidy* se han reorganizado los datos de manera que todos los valores medidos en una misma unidad ocupan una única columna. Para ello se han creado dos nuevas columnas correspondientes a dos nuevas variables, **Part**, que indica si se trata de **Sepal** o **Petal**, y **Measure**, que indica si se ha medido **Length** o **Width**.

Librería **tidyr**: organización de datos en tablas

Para ayudarnos en la tarea de estructurar los datos según este formato *tidy*, en R se dispone de la librería **tidyr**. Consulta el [cheatsheet data-import.pdf](#) para ver cuáles son las funciones más importantes de esta librería. En esta práctica veremos sobretudo las siguientes:

- **gather(data, key, value, ...)**: permite agrupar o juntar varias columnas en una sola en función de una misma clave (**key**) que toma los valores de los encabezados de las columnas agrupadas, y una columna **value** toma los valores de sus celdas.
- **spread(data, key, value, ...)**: hace lo contrario que **gather**, divide o esparce los valores que toma una variable dada por **key/value** como nuevas columnas cuyos nombres son los valores de **key**.
- **separate(data, col, into, ...)**: separa los contenidos que hay en una columna en varias columnas sin cambiar el número de observaciones (no confundir con **spread**).
- **unite(data, col, ...)**: hace lo contrario de **separate**, junta los contenidos de varias columnas (no confundir con **gather**).

Aunque **gather** y **spread** se siguen utilizando, existen dos nuevas funciones de **tidyr** que tienen un comportamiento similar:

- **pivot_longer(data, cols, names_to, values_to)**: condensa la información de las columnas **cols** en dos, dadas por los valores de sus encabezados (**names_to**), y los contenidos de sus celdas (**values_to**).

- `pivot_wider(data, names_from, values_from)`: es el caso inverso: extiende las columnas dadas por `names_from` y `values_from` a tantas columnas como valores tome la variable `names_from`.

También son interesantes las siguientes:

- `drop_na(data, ...)`: elimina las filas que contienen NA's.
- `fill(data, ...)`: sustituye los NA's por el valor más reciente no NA.
- `replace_na(data, replace=list(), ...)`: reemplaza los NA's con los valores suministrados.

Volviendo al ejemplo del dataset IRIS, veamos cómo podemos usar `gather` y `separate` para convertir el conjunto original `iris` en el conjunto `iris.tidy`.

```
library(tidyr)
# Primero agrupamos todas las unidades
# Agrupamos todas las columnas con datos excepto Species, que está bien
# iris.t <- gather(data=iris, key="Part", value="Value", 1:4)
iris.t <- gather(iris, Part, Value, -Species)
head(iris.t)
```

	Species	Part	Value
1	setosa	Sepal.Length	5.1
2	setosa	Sepal.Length	4.9
3	setosa	Sepal.Length	4.7
4	setosa	Sepal.Length	4.6
5	setosa	Sepal.Length	5.0
6	setosa	Sepal.Length	5.4

```
# Luego separamos la variable Part en dos, Part y Measure
iris.t <- separate(iris.t, Part, into=c("Part", "Measure"))
head(iris.t)
```

	Species	Part	Measure	Value
1	setosa	Sepal	Length	5.1
2	setosa	Sepal	Length	4.9
3	setosa	Sepal	Length	4.7
4	setosa	Sepal	Length	4.6
5	setosa	Sepal	Length	5.0
6	setosa	Sepal	Length	5.4

Ahora ya tiene formato *tidy*. Cada columna contiene una variable, y cada fila una observación.

Podemos hacer lo mismo con `pivot_longer`. Observa como cambia la ordenación de las filas.

```
iris.t.2 <- pivot_longer(iris, col=-Species, names_to="Part", values_to="Value")
head(iris.t.2)
```

```
# A tibble: 6 x 3
  Species Part      Value
  <fct>   <chr>    <dbl>
1 setosa Sepal.Length  5.1
2 setosa Sepal.Width   3.5
3 setosa Petal.Length  1.4
4 setosa Petal.Width   0.2
5 setosa Sepal.Length  4.9
6 setosa Sepal.Width    3
```

```
# Luego separamos la variable Part en dos, Part y Measure
iris.t.2 <- separate(iris.t.2, Part, into=c("Part", "Measure"))
head(iris.t.2)
```

```
# A tibble: 6 x 4
  Species Part Measure Value
  <fct>    <chr> <chr>   <dbl>
1 setosa Sepal Length    5.1
2 setosa Sepal Width    3.5
3 setosa Petal Length    1.4
4 setosa Petal Width    0.2
5 setosa Sepal Length    4.9
6 setosa Sepal Width     3
```

Librería magrittr: ceci n'est pas une pipe

Como habrás observado, para conseguir formatear el conjunto `iris` hemos tenido que aplicar dos operaciones, `gather` y `separate`. Esto nos ha obligado a tener que crear una variable, `iris.t`, donde almacenar el resultado de `gather`, para luego pasárselo a `separate`. También podríamos haber hecho algo como:

```
iris.t <- separate(gather(iris, Part, Value, -Species),
                  Part, into=c("Part", "Measure"))
head(iris.t)
```

```
Species Part Measure Value
1 setosa Sepal Length    5.1
2 setosa Sepal Length    4.9
3 setosa Sepal Length    4.7
4 setosa Sepal Length    4.6
5 setosa Sepal Length    5.0
6 setosa Sepal Length    5.4
```

Pero queda algo confuso, y aún lo es más cuando en lugar de dos operaciones hemos de realizar varias seguidas.

Una alternativa es usar la librería `magrittr` que define el operador `%>%` que permite encadenar este tipo acciones. El concepto es que los datos pasan de una función a otra como a través de una tubería (*pipe* en inglés) de funciones. Veamos cómo queda nuestro ejemplo:

```
iris.t <- gather(iris, Part, Value, -Species) %>% separate(Part, into=c("Part", "Measure"))
iris.t %>% head
```

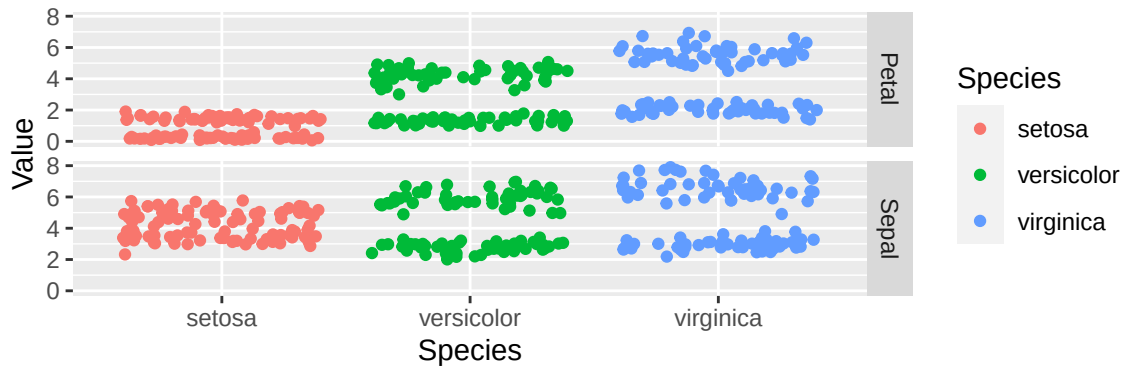
```
Species Part Measure Value
1 setosa Sepal Length    5.1
2 setosa Sepal Length    4.9
3 setosa Sepal Length    4.7
4 setosa Sepal Length    4.6
5 setosa Sepal Length    5.0
6 setosa Sepal Length    5.4
```

Fíjate que no hace falta especificar el primer argumento en las funciones `gather`, `separate` y `head`. El operador `%>%` automáticamente lo hace por nosotros.

Ejemplo de visualización

¿Por qué es mejor el formato *tidy* que el original? Una razón es que algunas librerías y funciones esperan encontrar los datos en formato *tidy*, entre ellas `ggplot2`. Por ejemplo, imagina que queremos representar los datos separados en función de la medida realizada, Sepal o Petal. Con los datos originales es complicado, puesto que esta información aparece distribuida en cuatro columnas. Con el conjunto *tidy* es inmediato.

```
library(ggplot2)
ggplot(iris.t) + aes(Species, Value, color=Species) +
  geom_jitter() + facet_grid(Part ~ .)
```



En los siguientes ejercicios veremos los ejemplos más típicos de datos desorganizados, o *messy data*:

- Las cabeceras de las columnas son variables en sí mismas, no nombres de variables.
- Más de una variable aparece en una sola columna.
- Las variables aparecen tanto en filas como en columnas.
- Varias unidades de observación están almacenadas en una única tabla.
- Una única unidad de observación está almacenada en varias tablas.

Pregunta: ¿qué tipo de problema(s) de los mencionados aquí presenta el conjunto IRIS original?

Las cabeceras de las columnas son variables en lugar de nombres de variables

El siguiente conjunto de datos muestra la relación entre los ingresos y la religión en EEUU.

```
print(load('data/pew_religion.Rdata'))
```

```
[1] "pew_religion"      "pew_religion_tidy"
```

```
head(pew_religion[, 1:10])
```

	religion	<\$10k	\$10-20k	\$20-30k	\$30-40k	\$40-50k	\$50-75k	\$75-100k	\$100-150k	>150k
1	Agnostic	27	34	60	81	76	137	122	109	84
2	Atheist	12	27	37	52	35	70	73	59	74
3	Buddhist	27	21	30	34	33	58	62	39	53
4	Catholic	418	617	732	670	638	1116	949	792	633
5	Don't know/refused	15	14	15	11	10	35	21	17	18
6	Evangelical Prot	575	869	1064	982	881	1486	949	723	414

Como se ve, las cabeceras son valores de una variable de tipo factor (los ingresos) en lugar de nombres de variables.

Este conjunto de datos tiene en realidad tres variables, que deberían estar en tres columnas: **religion**, **ingresos** y **frecuencia** (esta última con el número de casos).

Para ver la utilidad de organizar los datos en formato *tidy*, imagina que quisiéramos realizar un gráfico donde se mostrara la cantidad de gente por cada intervalo de ingresos.

¿Cómo lo harías con el conjunto original?

Fíjate que este gráfico es mucho más fácil de realizar si en lugar de tener los intervalos de ingresos repartidos en columnas los tuviéramos como una variable.

Para convertir este conjunto a formato *tidy* tendremos que agrupar todas las columnas excepto la de **religion** en dos, una llamada **income** con los intervalos de ingresos, y otra llamada **freq** con el número de casos para cada **income**. Como ya hemos visto en el ejemplo del conjunto IRIS, esto se consigue con **gather**.

Ejercicio 1 Convierte el dataset `pew_religion` a formato *tidy* usando `gather`. La columna `income` debe contener una variable de tipo `factor` ordenado, el valor `Don't know/refused` será el nivel más alto.

Otro ejemplo usando el conjunto de datos `Billboard` que muestra cuando una canción entró en el top 100.

```
load('data/billboard.Rdata')
head(raw[, 1:12])
```

	year	artist	track	time	date.entered	wk1	wk2	wk3	wk4	wk5	wk6	wk7
1	2000	2 Pac	Baby Don't Cry	4:22	2000-02-26	87	82	72	77	87	94	99
2	2000	2Ge+her	The Hardest Part Of ...	3:15	2000-09-02	91	87	92	NA	NA	NA	NA
3	2000	3 Doors Down	Kryptonite	3:53	2000-04-08	81	70	68	67	66	57	54
4	2000	3 Doors Down	Loser	4:24	2000-10-21	76	76	72	69	67	65	55
5	2000	504 Boyz	Wobble Wobble	3:35	2000-04-15	57	34	25	17	17	31	36
6	2000	98? Give Me	Just One Nig...	3:24	2000-08-19	51	39	34	26	26	19	2

La posición (*ranking*) de cada canción en cada semana después de haber entrado en el Top 100 se almacena en las columnas `wk1` a `wk76` (ver `str(raw)`). Si un tema sale del Top 100 antes de 76 semanas se rellenan los datos con `NA`'s.

Para obtener la versión *tidy* hemos de juntar todas las columnas desde `wk1` a `wk76` asignando como clave (`key`) una variable de nombre `week`, como valor (`value`) otra variable de nombre `rank`.

Ejercicio 2 Obtén la versión *tidy* de este conjunto de datos usando `pivot_longer`.

Consejo: puedes eliminar el prefijo `wk` de las semanas mediante el argumento `names_prefix`. Opcionalmente puedes crear una columna `date` a partir de `date.entered`, sumando las semanas transcurridas desde la fecha de entrada en lista.

Varias variables en una misma columna

El conjunto de datos `tuberculosis`, de la *World Health Organization*, presenta este tipo de desordenación (además de la vista en el apartado anterior). Muestra el número de casos de tuberculosis por país, año y grupo demográfico. Estos grupos demográficos se muestran en los nombres de las columnas como `m` o `f` (sexo) y la edad (0-14, 15-24, 25-34, 35-44, 45-54, 55-64, 65- y desconocido (`u`)).

```
load('data/tb.Rdata')
head(raw[, 1:16])
```

	country	year	m014	m1524	m2534	m3544	m4554	m5564	m65	mu	f014	f1524	f2534	f3544	f4554	f5564
11	AD	2000	0	0	1	0	0	0	0	NA	NA	NA	NA	NA	NA	NA
37	AE	2000	2	4	4	6	5	12	10	NA	3	16	1	3	0	0
61	AF	2000	52	228	183	149	129	94	80	NA	93	414	565	339	205	99
88	AG	2000	0	0	0	0	0	0	1	NA	1	1	1	0	0	0
137	AL	2000	2	19	21	14	24	19	16	NA	3	11	10	8	8	5
166	AM	2000	2	152	130	131	63	26	21	NA	1	24	27	24	8	8

Podemos ver como efectivamente en una columna tenemos dos variables juntas, el sexo y el rango de edad. Por ejemplo, en `m014` la `m` hace referencia al sexo y `014` al rango de edad, dos variables que deberían estar separadas.

Para pasar a formato *tidy* necesitamos realizar dos operaciones en este caso. En la primera juntaremos todas las columnas `m...` y `f...` y los conteos (los llamaremos `casos`). En la segunda, separaremos las columnas con los datos de tipo `m014` en dos columnas, `sexo` y `edad`.

Ejercicio 3 Convierte el conjunto de datos `TB` en *tidy* utilizando `pivot_longer` sin emplear `separate`.

Variables almacenadas tanto en filas como en columnas

Esta es la forma más complicada de datos desordenados, y es lo que ocurre en el conjunto de datos del *Global Historical Climatology Network*. Se muestra el tiempo para la estación MX17004, en México, durante 5 meses del año 2010.

```
load('data/weather.Rdata')
head(raw[, 1:20])
```

	id	year	month	element	d1	d2	d3	d4	d5	d6	d7	d8	d9	d10	d11	d12	d13	d14	d15	d16
1	MX17004	2010	1	tmax	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
2	MX17004	2010	1	tmin	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
3	MX17004	2010	2	tmax	NA	27.3	24.1	NA	NA	NA	NA	NA	NA	NA	29.7	NA	NA	NA	NA	NA
4	MX17004	2010	2	tmin	NA	14.4	14.4	NA	NA	NA	NA	NA	NA	NA	13.4	NA	NA	NA	NA	NA
5	MX17004	2010	3	tmax	NA	NA	NA	NA	32.1	NA	NA	NA	NA	34.5	NA	NA	NA	NA	NA	31.1
6	MX17004	2010	3	tmin	NA	NA	NA	NA	14.2	NA	NA	NA	NA	16.8	NA	NA	NA	NA	NA	17.6

Como se observa, además de las variables `id`, `year` y `month`, tiene variables en las columnas `d1` a `d31` que deberían corresponder a una única variable `dia`, y en las filas, `tmin` y `tmax` deberían estar como columnas. Fíjate además que la columna `element` no es una variable, sino que almacena nombres de variables.

Para obtener la versión *tidy* primero juntamos todos los días en una variable `dia`. Luego tendremos que separar la columna `element` en dos columnas, `tmax` y `tmin`. Esto último lo lograremos con la función `pivot_wider` de `tidyr`, que no hemos utilizado hasta ahora.

Ejercicio 4 Obtén la versión *tidy* del conjunto de datos `weather` utilizando las funciones `pivot_longer` y `pivot_wider`. Además cambiaremos las columnas `year`, `month` y `dia` por una única variable `fecha`, usando una función adecuada de la librería `lubridate`. Para que la operación `pivot_wider` no dé error tendremos que eliminar los NA's cuando usemos `pivot_longer`.

Varias unidades de observación en una sola tabla

Volvemos al ejemplo del conjunto `Billboard` para ilustrar este caso. Si nos fijamos en la versión *tidy* que hemos obtenido en el ejercicio 2 vemos como `artist` y `time` se repiten para cada canción (`track`) y semana. Para evitar estas repeticiones hemos de dividir el dataframe en dos, uno de canciones donde almacenaremos el artista, el nombre de la canción y el tiempo, y otro con el *ranking* de canciones que nos dará la posición en el Top 100, la canción y la semana. A esta división de los datos en varias tablas para evitar duplicados se le llama *normalización*, y es habitual cuando estamos trabajando con datos almacenados en bases de datos, si bien no lo es tanto en el campo del análisis de datos, donde muchas veces para analizarlos se requiere volver a juntar de nuevo los datos en una única tabla.

Ejercicio 5 Separa los datos *tidy* que teníamos en el ejercicio 2 en dos tablas (data frames) tal y como se describe en el párrafo anterior y empleando la función `unique` para eliminar repeticiones.

Opcionalmente se puede crear un identificador, `id`, asociado al nombre de la canción (`track`), y en la tabla de ranking solo usaremos este identificador en lugar del nombre de la canción.

Una única unidad de observación almacenada en varias tablas

También nos podemos encontrar casos donde las observaciones están distribuidas en varias tablas. El procedimiento general para abordar estos casos es:

1. Leer todos los ficheros en una lista de tablas.
2. Añadir una nueva columna a cada tabla con el nombre del fichero original.
3. Combinar todas las tablas en una.

Esto se puede hacer fácilmente con las librerías `plyr` y `dplyr` que veremos en la siguiente práctica.

Otro conjunto messy...

Ejercicio 6 En la librería *tidyr* hay varios conjuntos de datos. **table1** es un conjunto *tidy* pero **table5** no lo es. Realiza las operaciones necesarias para obtener **table1** a partir de **table5**. También deben coincidir los tipos de datos de las columnas.

Entrega de ejercicios

Al finalizar la práctica debes completar el cuestionario del aula virtual. Debes subir a aula virtual el fichero RMarkdown y el compilado HTML con las soluciones a los ejercicios planteados. **No subas un fichero comprimido, sube los ficheros .Rmd y .HTML separados.**